Metaplex 代币元数据

Solana Token Metadata Notes

本讲义围绕 **Metaplex Token Metadata Program** 展开，覆盖：

为什么需要元数据、Metadata PDA、核心字段、Metaplex SDK 创建与更新、
链下 JSON 规范、存储方案选择、与 Token-2022 内置元数据的差异、以及常见设计判断。

学习笔记 / Complete Notes

2026 年 3 月 22 日

目录

1 章节导读 **3**

2 为什么需要元数据 **3**
2.1 原生 SPL Token 只保存什么信息 . . . 3
2.2 为什么钱包不能只靠 Mint 地址判断代币 . . . 4

3 Metaplex 的核心思路 **4**
3.1 独立 Metadata Account . . . 4
3.2 为什么要用 PDA 绑定 . . . 5

4 Metadata Account 的核心字段 **5**
4.1 结构总览 . . . 5
4.2 最重要的三个字段 . . . 6
4.3 为什么 URI 这么关键 . . . 6

5 权限控制：Update Authority 与可变性 **6**
5.1 谁可以更新元数据 . . . 6
5.2 isMutable 的意义 . . . 7

6 使用 Metaplex SDK 创建带元数据的代币 **7**
6.1 安装依赖 . . . 7
6.2 创建带 Metaplex 元数据的代币 . . . 7
6.3 这段代码的执行逻辑 . . . 8
6.4 参数逐项解释 . . . 9

7 更新元数据 **9**
7.1 什么时候可以更新 . . . 9
7.2 更新示例 . . . 9
7.3 这段更新代码在做什么 . . . 10

8 链下元数据 JSON 规范 **10**
8.1 JSON 文件示例 . . . 11
8.2 每个字段的用途 . . . 11
8.3 为什么图片通常不直接上链 . . . 11

9 链下存储方案选择 **12**
9.1 常见方案比较 . . . 12
9.2 如何判断该选哪一种 . . . 12

10 Metaplex 与 Token-2022 内置元数据的关系 **12**
10.1 两者不是“谁绝对淘汰谁” . . . 12
10.2 差异直观理解 . . . 13
10.3 实际选择建议 . . . 13

11 常见误区与易错点 **13**
11.1 误区一：有 name 和 symbol 就够了 13
11.2 误区二：Metadata 一定完全上链 13
11.3 误区三：updateAuthority 不重要 14
11.4 误区四：所有 Fungible Token 都必须用 Metaplex 14
11.5 误区五：URI 能打开就一定没问题 14

12 本章最重要的 10 条结论 **14**

13 推荐的学习顺序 **14**
13.1 第一步：先理解 Mint 和 Metadata 是两层结构 14
13.2 第二步：手写一遍 Metadata PDA 的推导逻辑 15
13.3 第三步：跑一次 createV1 示例 15
13.4 第四步：再练一次 updateV1 15
13.5 第五步：最后再比较 Token-2022 元数据扩展 15

1 章节导读

这一章的重点不是“怎么把三个字符串写上链”这么简单，而是要理解：**为什么 Solana 生态需要额外的 Metadata Program、它和 Mint 账户是什么关系、为什么 URI 还要再指向链下 JSON、以及项目在 Token-2022 和 Metaplex 之间该如何选择**。

在实际使用中，Metaplex 元数据之所以非常重要，原因通常包括：

1. 钱包和区块浏览器需要人类可读名称，而不是只显示 Mint 地址。
2. NFT 市场和索引器普遍围绕 Metaplex 元数据生态构建。
3. 单靠 SPL Mint 账户本身，无法存储完整展示信息。
4. 图片、描述、属性等大字段不适合直接全部写进基础 Token 账户结构。
5. 项目方往往还需要可更新、可冻结、可验证的元数据管理机制。

一句话先记住：**Mint 账户更像“代币的数学与权限核心”，Metadata Account 更像“代币的说明书与展示层”**。

2 为什么需要元数据

2.1 原生 SPL Token 只保存什么信息

SPL Token 的 Mint 账户主要存的是代币运行所需的基础状态，例如：

- 总供应量（Supply）
- 小数位数（Decimals）
- Mint Authority
- Freeze Authority

这些字段足够支持：

- 增发（mint）
- 转账（transfer）
- 销毁（burn）
- 冻结与解冻（freeze / thaw）

但是，它并不能告诉钱包：

- 这个代币叫什么名字
- 它的符号是什么
- 它有没有 Logo

- 它的项目描述是什么
- 它是否属于某个 NFT 集合

如果没有元数据，钱包通常只能显示一串地址和余额。对用户来说，这样的资产几乎没有可读性，也不利于品牌识别与交互体验。

2.2 为什么钱包不能只靠 Mint 地址判断代币

因为链上地址本身只是一个公钥，不能天然表达语义。**地址是机器可识别的标识，不是人类可识别的品牌信息**。

例如，钱包若想显示：

- `USD Coin`
- `USDC`
- 对应的蓝色圆形 Logo
- 项目描述、官网链接

它就必须从某个额外的数据源读取这些信息，而这个“额外层”就是元数据体系。

可以把它理解成：
- Mint 账户：像商品条码系统中的“内部编码”
- Metadata：像货架上的商品名称、包装图案、说明标签

没有后者，用户几乎无法直观识别资产。

3 Metaplex 的核心思路

3.1 独立 Metadata Account

Metaplex 的设计不是把所有展示信息直接塞进 Mint 账户，而是：

- 让 Mint 账户继续负责 Token 的基础逻辑
- 再创建一个独立的 Metadata Account
- 用 PDA 将该 Metadata Account 和 Mint 绑定起来

所以 Metaplex 的核心思想是：**把“代币逻辑”和“代币展示信息”分开管理**。

这种结构有几个优点：

1. Token Program 不需要频繁扩展自身结构。
2. Metadata 可以包含更复杂的展示字段。
3. 钱包、市场、索引器都能按照统一规范读取。

4. 对 NFT 来说，还能自然扩展创作者、版税、集合等信息。

3.2 为什么要用 PDA 绑定

PDA（Program Derived Address）可以让 Metadata Account 的地址由固定规则推导出来，而不是随机生成。

这样一来：

- 同一个 Mint 总能找到对应的 Metadata 地址
- 外部程序不需要“到处搜索”元数据账户
- 地址推导规则公开透明，生态可以统一读取

Listing 1: Metadata PDA 推导逻辑示意

```
Metadata PDA = findProgramAddress(
    ["metadata", METADATA_PROGRAM_ID, MINT_ADDRESS],
    METADATA_PROGRAM_ID
)
```

你可以把 PDA 理解成“按规则算出来的官方附属地址”。对于某个 Mint，只要大家都遵循同一套推导规则，就会得到同一个 Metadata 地址。

4 Metadata Account 的核心字段

4.1 结构总览

虽然内部序列化结构比较复杂，但学习时可以先把它理解成下面这组核心字段：

Listing 2: Metadata 核心字段示意

```
interface Metadata {
    // 基础信息
    name: string;          // 代币名称（最多 32 字符）
    symbol: string;        // 代币符号（最多 10 字符）
    uri: string;           // 指向链下 JSON 的 URL

    // 权限控制
    updateAuthority: PublicKey;  // 谁可以修改元数据
    isMutable: boolean;          // 是否允许修改

    // 版税（主要用于 NFT）
    sellerFeeBasisPoints: number;  // 二次销售版税（如 500 = 5%）
    creators: Creator[];           // 创作者及分成比例

    // 集合（主要用于 NFT）
    collection: Collection | null;
    uses: Uses | null;
```

```
}
```

4.2 最重要的三个字段

在初学阶段，你最应该先抓住这三个：

- `name`
- `symbol`
- `uri`

其中：

- `name` 决定用户在钱包里看到的代币名称
- `symbol` 决定简写符号
- `uri` 则是通往更完整展示信息的入口

真正的图片、长描述、attributes、external_url 等往往不是直接塞在链上 Metadata Account 里，而是放在 `uri` 指向的链下 JSON 文件中。

4.3 为什么 URI 这么关键

如果你把图片、长文本、属性数组全部直接存到链上：

- 账户空间会更大
- 租金更高
- 结构更笨重
- 更新和维护成本也更高

因此 Metaplex 采用的是非常典型的设计：

链上保存“入口信息”，链下保存“完整展示内容”

5 权限控制：Update Authority 与可变性

5.1 谁可以更新元数据

Metaplex 元数据并不是任何人都能改。通常只有 `updateAuthority` 才有权限更新该 Metadata。

这代表它可以控制：

- 改名称
- 改符号
- 改 URI

- 改其他展示信息

如果项目方保留 `updateAuthority`，就表示它在未来仍然可能修改元数据。这对项目运营是灵活性，但对用户来说也意味着信任问题。

5.2 isMutable 的意义

`isMutable` 表示元数据之后是否还能继续改。

- `true`：后续允许更新
- `false`：通常意味着进入定稿状态

在 NFT 语境里，很多项目会特别强调：

- 元数据是否已经 reveal
- reveal 后是否还允许改动
- 是否已经永久锁定

所以判断一个资产的“可信展示状态”时，不只是看名字和图片，还要看：**updateAuthority 还在不在、isMutable 是否仍为 true**。

6 使用 Metaplex SDK 创建带元数据的代币

6.1 安装依赖

首先安装所需依赖：

Listing 3: 安装 Metaplex SDK 相关依赖

```
npm install @metaplex-foundation/mpl-token-metadata \
            @metaplex-foundation/umi \
            @metaplex-foundation/umi-bundle-defaults
```

6.2 创建带 Metaplex 元数据的代币

下面这个示例展示：

- 如何初始化 Umi
- 如何载入本地钱包
- 如何创建新的 Mint
- 如何在创建代币的同时创建 Metadata

Listing 4: create-token-with-metaplex.ts

```
import { createUmi } from '@metaplex-foundation/umi-bundle-defaults';
import {
```

```
    mplTokenMetadata,
    createV1,
    TokenStandard
} from '@metaplex-foundation/mpl-token-metadata';
import {
    generateSigner,
    percentAmount,
    keypairIdentity
} from '@metaplex-foundation/umi';
import { loadWallet } from './config';

async function createTokenWithMetaplex() {
    // 初始化 Umi
    const umi = createUmi('https://api.devnet.solana.com')
        .use(mplTokenMetadata());

    // 加载钱包
    const wallet = loadWallet();
    const umiKeypair = umi.eddsa.createKeypairFromSecretKey(wallet.secretKey);
    umi.use(keypairIdentity(umiKeypair));

    // 生成 Mint 密钥对
    const mint = generateSigner(umi);

    console.log('创建带 Metaplex 元数据的代币...');

    // 创建代币 (同时创建 Mint 和 Metadata)
    await createV1(umi, {
        mint,
        name: 'Solana University Token',
        symbol: 'SOLU',
        uri: 'https://arweave.net/your-metadata-json',
        sellerFeeBasisPoints: percentAmount(0),  // 无版税
        decimals: 6,
        tokenStandard: TokenStandard.Fungible,
    }).sendAndConfirm(umi);

    console.log('创建成功! ');
    console.log('Mint 地址:', mint.publicKey);
}

createTokenWithMetaplex();
```

6.3 这段代码的执行逻辑

建议你按以下顺序理解这段代码:

1. 用 `createUmi(...)` 连接 Devnet。

2. 用 `mplTokenMetadata()` 挂载 Metadata Program 支持。
3. 用本地私钥导入身份，让后续交易能签名。
4. 生成一个新的 Mint signer，准备创建新代币。
5. 调用 `createV1(...)`，一次性把 Mint 与 Metadata 都建立起来。
6. 发送并确认交易。

这里最重要的思想不是“API 名字怎么背”，而是要知道：**createV1 背后做的事情，本质上是按照 Metaplex 规则建立 Mint 与 Metadata 的配套结构**。

6.4 参数逐项解释

参数	说明
`mint`	新生成的 Mint signer，对应将要创建的代币。
`name`	代币名称，供钱包和界面展示。
`symbol`	代币符号，例如 SOLU、USDC。
`uri`	指向链下 JSON 文件的 URL。
`sellerFeeBasisPoints`	二级销售版税，NFT 常见；可替代性代币常设为 0。
`decimals`	小数位数；可替代性代币常为 6、8、9 等。
`tokenStandard`	资产类型标准；本例使用 `TokenStandard.Fungible`。

这里的 `sellerFeeBasisPoints` 虽然在 Fungible Token 中常常设为 0，但它仍然保留了与 NFT 生态兼容的字段结构。

7 更新元数据

7.1 什么时候可以更新

只有满足以下条件时，元数据通常才可以继续更新：

- 当前签名者是 `updateAuthority`
- 该 Metadata 仍允许更新（例如 `isMutable = true`）

7.2 更新示例

Listing 5: update-metadata.ts

```
import { createUmi } from '@metaplex-foundation/umi-bundle-defaults';
import {
    mplTokenMetadata,
    updateV1,
    fetchMetadataFromSeeds
} from '@metaplex-foundation/mpl-token-metadata';
```

```
import { publicKey, keypairIdentity } from '@metaplex-foundation/umi';
import { loadWallet } from './config';

async function updateMetadata(mintAddress: string) {
    const umi = createUmi('https://api.devnet.solana.com')
        .use(mplTokenMetadata());

    const wallet = loadWallet();
    const umiKeypair = umi.eddsa.createKeypairFromSecretKey(wallet.secretKey);
    umi.use(keypairIdentity(umiKeypair));

    const mint = publicKey(mintAddress);

    // 获取当前元数据
    const metadata = await fetchMetadataFromSeeds(umi, { mint });
    console.log('当前名称:', metadata.name);

    // 更新元数据
    await updateV1(umi, {
        mint,
        data: {
            ...metadata,
            name: 'Updated Token Name',
            symbol: 'NEW',
        },
    }).sendAndConfirm(umi);

    console.log('更新成功! ');
}
```

7.3 这段更新代码在做什么

这段代码的逻辑可以拆成四步：

1. 连接网络并加载身份。
2. 根据 Mint 地址推导并抓取当前 Metadata。
3. 读取已有字段，避免更新时把原信息全部覆盖丢失。
4. 只修改想变动的字段，再提交更新交易。

更新元数据时，一个常见错误是：直接只传新名字和新符号，却没有保留其他原字段，导致结构不完整或意外覆盖。所以实务上常会先读旧数据，再在其基础上局部修改。

8 链下元数据 JSON 规范

8.1 JSON 文件示例

Listing 6: 链下元数据 JSON 示例

```
{
    "name": "Solana University Token",
    "symbol": "SOLU",
    "description": "Solana 大学官方学习激励代币，用于奖励优秀学员",
    "image": "https://arweave.net/your-logo-image",
    "external_url": "https://academy.soldevcamp.com",
    "attributes": [],
    "properties": {
        "category": "currency",
        "files": [
            {
                "uri": "https://arweave.net/your-logo-image",
                "type": "image/png"
            }
        ]
    }
}
```

8.2 每个字段的用途

字段	作用
`name`	展示名称，通常与链上 Metadata 的 name 保持一致。
`symbol`	简写符号。
`description`	更长的项目说明文字。
`image`	钱包或市场常用于展示封面图 / Logo。
`external_url`	项目官网或进一步资料入口。
`attributes`	NFT 中常见属性数组；Fungible Token 可为空。
`properties`	附加文件、分类等更细节的展示信息。

请特别记住：**链上 Metadata 的 URI 只是“入口地址”，真正让界面变得丰富的，是 URI 背后的 JSON 内容**。

8.3 为什么图片通常不直接上链

主要原因包括：

- 图片文件太大
- 链上存储成本高
- 更新和分发不方便
- 钱包和浏览器更习惯用 URL 拉取资源

所以常见做法是：

- 图片放在 Arweave / IPFS / 云存储
- JSON 中填入图片 URL
- 链上 Metadata 再通过 URI 指到 JSON

9 链下存储方案选择

9.1 常见方案比较

方案	特点	适用场景
Arweave	永久存储，一次付费	NFT、长期项目、重视持久性
IPFS + Pinata	去中心化存储，但通常需要 pin 服务维护	一般项目、希望兼顾开放性
AWS S3 / 云存储	中心化，但便宜、简单、稳定	测试环境、内部项目、开发阶段

9.2 如何判断该选哪一种

- 如果你做的是高价值 NFT 或长期收藏品，通常优先考虑 **Arweave**。
- 如果你想保留一定开放性，同时接受额外维护，**IPFS + Pinata** 是常见选择。
- 如果你只是课堂作业、内部测试、Devnet 演示，**AWS S3 或普通云存储**往往最省事。

开发阶段最常见的失误之一，是先把 JSON 放在一个会变动、会失效、或权限不稳定的 URL 上，结果钱包后来无法正常读取图片或描述。

10 Metaplex 与 Token-2022 内置元数据的关系

10.1 两者不是“谁绝对淘汰谁”

很多初学者会误以为：

“既然 Token-2022 已经支持元数据，那 Metaplex 就不重要了。”

这其实不准确。

更准确的理解是：**Token-2022 提供了内置元数据能力，但 Metaplex 仍然在生态兼容性和 NFT 场景中占据非常重要的位置。**

10.2 差异直观理解

比较点	Metaplex Metadata	Token-2022 内置 Metadata
存储方式	独立 Metadata Account	扩展挂在 Mint / 关联结构上
生态成熟度	非常成熟，NFT 领域主流	新一代扩展方案，支持逐步增长
NFT 适配度	很强	可做，但生态习惯未必一致
Fungible Token 轻量方案	相对更重	更轻量、更原生
钱包 / 市场兼容性	普遍较好	视具体生态支持情况而定

10.3 实际选择建议

- 做 **NFT / 收藏品 / 需要市场兼容**：优先理解并考虑 Metaplex。
- 做 **带扩展功能的 Fungible Token**：可以认真考虑 Token-2022。
- 做 **教学练习**：两者都值得学，因为它们代表 Solana 生态两种非常重要的设计路线。

不要把“技术上能做”直接等同于“生态上最好用”。链上开发里，兼容性、钱包支持、索引器支持、市场采用度，常常和代码功能本身一样重要。

11 常见误区与易错点

11.1 误区一：有 name 和 symbol 就够了

不够。因为用户界面通常还需要：

- 图片
- 描述
- 官网
- 属性
- 文件资源

这些往往要靠 URI 对应的 JSON 来补足。

11.2 误区二：Metadata 一定完全上链

并不一定。Metaplex 的经典设计就是：

- 链上放索引与关键字段
- 链下放丰富展示内容

11.3 误区三：updateAuthority 不重要

实际上非常重要。因为它决定项目是否仍可改名、改图、改描述。对资产可信度判断来说，这是核心字段之一。

11.4 误区四：所有 Fungible Token 都必须用 Metaplex

也不一定。如果你只是想做一个轻量但带扩展的 Fungible Token，Token-2022 的内置元数据可能更适合。

11.5 误区五：URI 能打开就一定没问题

还不一定。因为还要检查：

- JSON 格式是否正确
- 图片链接是否可访问
- CORS / 网关 / pin 服务是否稳定
- 钱包或索引器是否接受该格式

12 本章最重要的 10 条结论

1. 原生 SPL Mint 账户主要负责代币的数学与权限信息，不负责完整展示层。
2. Metaplex 通过独立 Metadata Account 来管理名称、符号、URI 等信息。
3. Metadata Account 和 Mint 之间通常通过 PDA 规则建立唯一绑定。
4. `name`、`symbol`、`uri` 是最先要理解的三个核心字段。
5. 真正丰富的展示内容，通常存放在 URI 指向的链下 JSON 文件中。
6. `updateAuthority` 决定谁可以改元数据，直接影响资产可信度判断。
7. `isMutable` 决定元数据是否还允许继续更新。
8. Metaplex 在 NFT 生态中仍然极其重要，并没有因为 Token-2022 出现就失去价值。
9. Token-2022 更适合某些扩展型 Fungible Token，而 Metaplex 在兼容性上依然很强。
10. 设计链上资产时，不能只看“能不能写代码实现”，还要看钱包、市场和生态支持。

13 推荐的学习顺序

13.1 第一步：先理解 Mint 和 Metadata 是两层结构

先彻底分清：

- Mint 不是展示层

- Metadata 不是余额账户
- 两者职责不同

13.2 第二步：手写一遍 Metadata PDA 的推导逻辑

即使先不深入底层实现，也要理解：

- 为什么 Metadata 地址是可推导的
- 为什么不是随机新建一个无规则账户

13.3 第三步：跑一次 createV1 示例

建议在 Devnet：

- 创建一个简单 Fungible Token
- 配一个可访问的测试 URI
- 看钱包或浏览器读取结果

13.4 第四步：再练一次 updateV1

然后测试：

- 改名字
- 改符号
- 改 URI

这样你会更直观理解 `updateAuthority` 的实际意义。

13.5 第五步：最后再比较 Token-2022 元数据扩展

等 Metaplex 的结构先吃透以后，再去比较：

- 为什么 Token-2022 要把元数据能力内建进去
- 两种路线在工程设计上各自解决了什么问题